

Android & iOS Native Screen Recorder

[Check online](#)

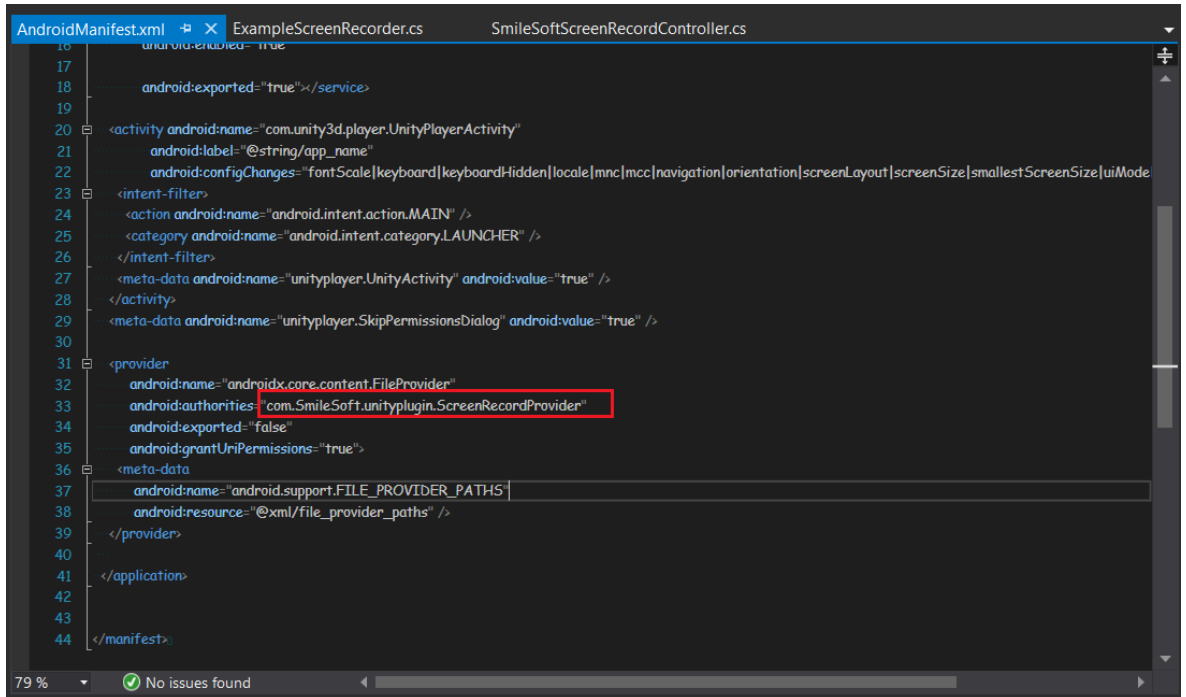
This plugin can record your gameplay without any issues. It uses Android native **MediaRecorder** API and **Android Foreground Service** for the record screens. So it will support from **Lollipop (5.1)** to newer versions even **Android 13(The latest One)** and above.

In iOS, it uses the iOS replayKit Library. It will support all iPhone, iPad, and iPod Touch devices with iOS 9.0 or above.

Instruction: After importing the project please drag **Screen Recorder** prefab in Scene Hierarchy from **Assets > SunShine Android Native Screen Recorder > Prefab > Screen Recorder**.

If you want to share the video using the native share dialog then please do the following settings. **(Only Android)**

1. Open the Android manifest file from **Assets > Plugins > Android > AndroidManifest.xml**. Now change the authority name with a unique one. We highly recommend you use the package name.



```
16 android:enabled="true"
17
18 android:exported="true"</service>
19
20 <activity android:name="com.unity3d.player.UnityPlayerActivity"
21 android:label="@string/app_name"
22 android:configChanges="fontScale|keyboard|keyboardHidden|locale|mnc|mcc|navigation|orientation|screenLayout|screenSize|smallestScreenSize|uiMode"
23 <intent-filter>
24 <action android:name="android.intent.action.MAIN" />
25 <category android:name="android.intent.category.LAUNCHER" />
26 </intent-filter>
27 <meta-data android:name="unityplayer.UnityActivity" android:value="true" />
28 </activity>
29 <meta-data android:name="unityplayer.SkipPermissionsDialog" android:value="true" />
30
31 <provider
32 android:name="androidx.core.content.FileProvider"
33 android:authorities="com.SmileSoft.unityplugin.ScreenRecordProvider"
34 android:exported="false"
35 android:grantUriPermissions="true">
36 <meta-data
37 android:name="android.support.FILE_PROVIDER_PATHS"
38 android:resource="@xml/file_provider_paths" />
39 </provider>
40
41 </application>
42
43
44 </manifest>
```

2. Again, Open **SmileSoftScreenRecordController.cs** from **Assets > SunShine Android Native Screen Recorder > Scripts > SmileSoftScreenRecordController.cs**. Now change the `_fileProvider` value which you set on the manifest file before.

```

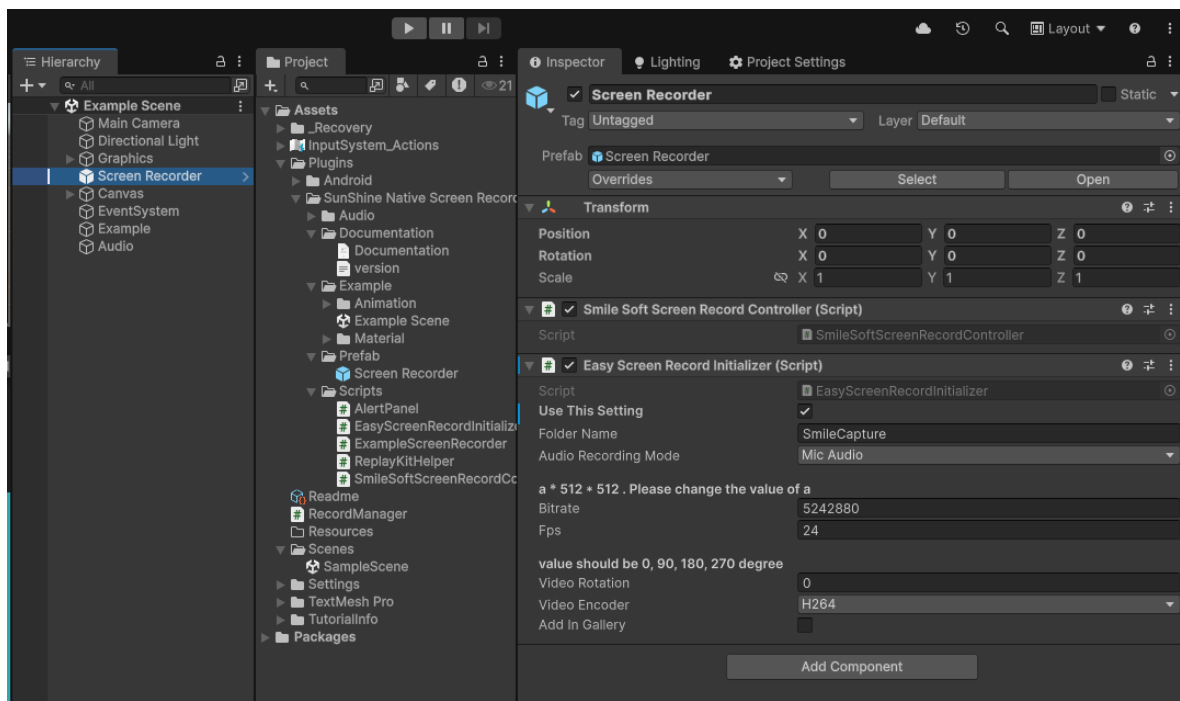
1 using System.Collections;
2 using System.Collections.Generic;
3 using System.IO;
4 using UnityEngine;
5
6 public class SmileSoftScreenRecordController : MonoBehaviour
7 {
8     public static SmileSoftScreenRecordController instance;
9
10    private string _fileProvider = "com.SmileSoft.unityplugin.ScreenRecordProvider";
11
12    private AndroidJavaObject screenRecorder;
13
14 }

```

You can also see this [video](#) For setting the file provider path value.

Now you are ready for calling the plugin API. The following functions are available in this plugin.

Easy Initializer: You will find a ready recorder initializer in the **ScreenRecorder** prefab. You can enable or disable this configuration from the Inspector using the **Use This Setting** toggle. This section allows you to easily adjust and fine-tune the recorder settings according to your needs.



Functions:

1. **Start Record:** `SmileSoftScreenRecordController.instance.StartRecording()`
2. **Stop Record:** `SmileSoftScreenRecordController.instance.StopRecording()` . In Android, It returns the recorded file path. in iOS devices, it will open the native preview window. From there you can edit, discard or save the recorded video in the camera roll even you can share it instantly with your sharing apps.

3. **Record Audio:** This plugin supports screen recording with or without audio, and the behavior varies slightly between Android and iOS.
 - **Android:** You can choose the audio source (Microphone or In-Game/System Audio).
 - **iOS:** Audio is always recorded from the **microphone** (system audio is not supported).

`SmileSoftScreenRecordController.instance.SetAudioRecordingMode(SmileSoftScreenRecordController.AudioRecordingMode.MicAudio);`

AudioRecordingMode (Enum) -

NoAudio – Records screen without audio

SystemAudio – Records in-game/system audio (Android only)

MicAudio – Records microphone audio (Android & iOS)

4. **Set File Destination:** `SmileSoftScreenRecordController.instance.SetVideoDestination(destination)`. Where the **destination** is a **String** type value, by default, the destination is the **External Storage** directory. **(Only Android)**
5. **Set File Name:** `SmileSoftScreenRecordController.instance.SetVideoName(fileName)`. Where **fileName** is a **String** type value. **(Only Android)**
6. **Folder Name:** You can create a new folder and save your video there. Just call `SmileSoftScreenRecordController.instance.SetStoredFolderName(folderName)` function. Here **folderName** is a **String** type value. **(Only Android)**
7. **Set Gallery Add Capabilities:** `SmileSoftScreenRecordController.instance.SetGalleryAddingCapabilities(canAddIntoGallery)`. Where **canAddintoGallery** is a **boolean** type value. If it is true then the video file will be added to the device gallery. By default this value is **true**. **(Only Android)**
8. **Bit Rate:** `SmileSoftScreenRecordController.instance.SetBitRate(bitrate)`. **Bitrate** is an **integer-type** parameter. A higher bitrate means high-quality video. Please try to set this value like - $a * 512 * 512$. change the value of a. **(Only Android)**
9. **Preview:** If you want to see the preview in the native video view then just call this function `SmileSoftScreenRecordController.instance.PreviewVideo(_recordedFilePath)`. Here **recordedFilePath** is a **string-type** parameter. For iOS, you do not have to pass the file path.
10. **Share Video:** You can also Share the video using the native share dialog. Just call `SmileSoftScreenRecordController.instance.ShareVideo(sssss)`. It takes three parameters and all of them are **strings**. The first one is the **filePath**, the second one is the **share message** and the third one is the **share title**. **(Only Android)**
11. **Video Size:** `SmileSoftScreenRecordController.instance.SetVideoSize(width, height)`. Both **width** and **height** are integer-type parameters. **(Only Android)**
12. **Video Encoder:** `SmileSoftScreenRecordController.instance.SetVideoEncoder(videoEncoder)`. Here **video encoder** is an integer-type parameter. Before setting this please look at the Android [Developer official](#) site for supporting encoders in different Android API versions. **(Only Android)**
13. **Video Rotation:** `SmileSoftScreenRecordController.instance.SetVideoRotation(Videorotation)`. Here **video rotation** is an integer-type parameter. Values should be 0, 90, 180, or 270. Before setting this please look at the Android [Developer official](#) site for supporting rotations in different encoders and different Android

API versions. **(Only Android)**

To get a clear idea please see the example scene from **Assets > SunShine Android Native Screen Recorder > Example > Example Scene. As**

**** Here 3-9 number functions are optional. You can use all of them or none. But please be sure that these (3-9) functions are called before calling the Start Record function. ****

Android CallBack Functions: You get the following callbacks in [SmileSoftScreenRecordController.cs](#) script.

OnRecordPermissionGranted - It will call when the Record permission popup opens and users accept/reject the permission.

Parameter: Status (string)

True - Accept permission.

False - Declined Permission

OnRecordStartStatus - It will call when a user tries to record.

Parameter: Status (string)

True - Record Start Successfully

False - Record Failed

iOS CallBack Functions: You get the following callbacks in [SmileSoftScreenRecordController.cs](#) script.

OnlosRecordStatus - It will call when the iOS Record will start or fail.

Parameter: Status (Boolean)

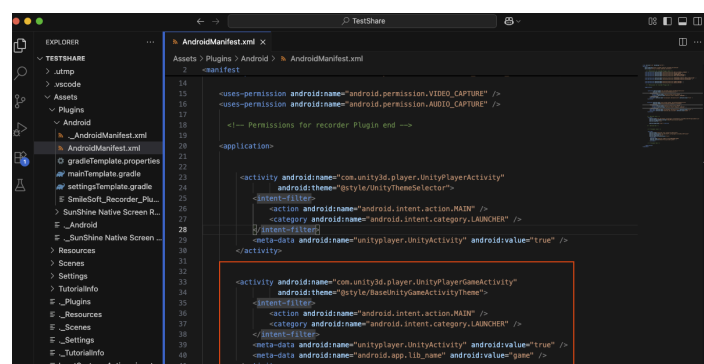
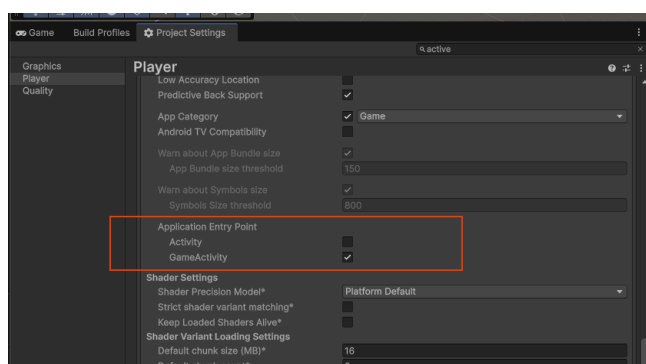
True - Successfully start recording

False - Failed to start recording

Troubleshoot -

Android - Check the Application Entry point from **Project Settings -> Select Android -> Other Settings -> Application Entry Point**. Make sure **Game Activity** is selected.

If you don't want to keep the **Game Activity** as the entry point, then remove the **UnityPlayerGameActivity** section from the activity part.



IOS - In iOS only **file path** and **message** parameters are supported. Others are set automatically. So you do not have to worry about other parameters.
In iOS you might see a (**dyld: Library not loaded: @rpath/libswiftCore.dylib**) error when you try to build it in an iOS device. For resolving this from Xcode set **Always Embed Swift Standard Libraries** value to **true**. You found this in the **Build Setting** tab.

